WHY RAG BREAKS & WHAT COMES NEXT

Context Rot, Production Reality, and the Evolution of Retrieval

Advanced AI Engineering Series
Class 7b: Production Retrieval Systems

CONTEXT ROT: THE HIDDEN KILLER

The 10,000th token is NOT treated as reliably as the 100th.

SOURCE: CHROMA RESEARCH (2025)
TESTED 18 FRONTIER LLMS

THE PROMISE

"Just give the model more context! 1M+ token windows allow RAG to retrieve everything and solve hallucination."

THE REALITY

Models do not read context uniformly. Performance degrades unpredictably as input length grows—even on simple tasks.

More tokens often = Worse answers.

WHY DOES CONTEXT ROT HAPPEN?

THE ATTENTION BUDGET

Self-attention is a **finite resource**. Every token computes scores against every other token in the sequence.

More tokens = Attention stretched thinner.

As sequence length grows, the model "forgets" to focus on critical information, leading to erratic and unreliable outputs.

TRANSFORMER REALITY

"Context must be treated as a resource with diminishing marginal returns." — Anthropic

> *"It's like asking someone to read War and Peace in one sitting, then answer a specific question."*

Attention scores become diluted across massive token counts.

THE DISTRACTORS PROBLEM

EXPERIMENT: NEEDLE IN A HAYSTACK (INFERENCE)

Question: "Which character has been to Dresden?"
Needle: "Yuki lives next to the Semper Opera House"
(Requires inference: Semper Opera House is in Dresden)

THE RESULT

Short context: **>90% Accuracy**

32K context: **Accuracy drops dramatically**

**Add 4 similar-but-wrong distractors?
Performance TANKS.**

THE INSIGHT

Coherent documents are **harder** than random text. Models follow narrative arcs instead of finding the needle.

YOUR NICELY FORMATTED DOCUMENTS MIGHT BE HURTING RETRIEVAL.

NAIVE RAG VS. PRODUCTION REALITY

THE TUTORIAL VERSION

1. Query → Embed
2. Search Vector DB
3. Get Top-K Chunks
4. Stuff into Prompt
5. Generate Answer

"Works great on demos with short, clean documents."

PRODUCTION CHALLENGES

DISTRACTORS

Top-K retrieval brings back similar but wrong chunks that confuse the model.

CONTEXT ROT

Stuffing all K chunks stretches attention thin and degrades accuracy.

PASSIVE MODEL

The LLM takes whatever noise you give it without any way to verify relevance.

YOUR RAG WORKS ON DEMOS. IT BREAKS ON PRODUCTION DATA.

HOW CHATGPT ACTUALLY MANAGES MEMORY

THE COMMON ASSUMPTION

> "ChatGPT remembers past conversations using RAG over chat history and vector databases."

Most developers assume a standard embedding-based retrieval system is at work behind the scenes.

THE PRODUCTION REALITY

NO VECTOR DATABASES.
NO RAG.
NO EMBEDDINGS SEARCH.

Reverse-engineering reveals a far more surgical approach to context management.

CHATGPT'S MEMORY ARCHITECTURE

Surgical context engineering over brute-force retrieval.

LAYER 1: SESSION METADATA

Device, location, subscription tier, and usage patterns. Injected once at session start.

LAYER 2: EXPLICIT FACTS

Long-term user preferences stored as text (e.g., "User prefers Python over JavaScript").

LAYER 3: CONVERSATION SUMMARIES

~15 lightweight digests of recent chats. Only includes user messages, not assistant replies.

LAYER 4: CURRENT SESSION

The full sliding window of the active conversation.

NO EMBEDDINGS. NO VECTOR SEARCH.

MAKING RAG WORK: ADVANCED TECHNIQUES

RAG ISN'T DEAD — BUT NAIVE RAG IS.

1. BETTER RETRIEVAL

HYBRID SEARCH

Combining BM25 keywords with semantic vector search.

RERANKING

Using cross-encoders to score candidate relevance.

QUERY TRANSFORMATION

Rewriting queries (HyDE) to improve match rates.

2. BETTER CHUNKING

SEMANTIC CHUNKING

Splitting documents by meaning rather than character count.

PARENT-CHILD CHUNKS

Retrieve small snippets, provide large context to LLM.

METADATA FILTERING

Narrowing search by date, source, or document type.

3. BETTER GENERATION

CITATION & ATTRIBUTION

Forcing the model to link every claim to a source chunk.

SELF-VERIFICATION

Checking if the answer is supported by the context.

CONTEXT ENGINEERING

Being surgical about what the model actually sees.

GOAL: BE SURGICAL. EVERY TOKEN ADDED IS A BET AGAINST ACCURACY.

HYBRID SEARCH

The Trade-off:

Semantic search misses exact matches (codes, IDs). Keyword search misses conceptual meaning.

THE SOLUTION: BM25 + VECTOR

KEYWORD (BM25) | **SEMANTIC (VECTOR)**

Run both searches simultaneously and merge results using **Reciprocal Rank Fusion (RRF)**.

```
Query: "Error 404 in user auth module"
 BM25: Finds exact "Error 404" strings.
Vector: Finds docs about "authentication".
```

Result: High precision AND high recall.

RERANKING: THE "SECOND PASS" FOR ACCURACY

STAGE 1: RETRIEVAL

FAST & SCALABLE

Use "cheap" methods like **Vector Search** or **BM25** to retrieve the top 100-500 candidates from millions of documents.

Goal: High Recall. Don't miss the answer, even if it's not at the very top.

STAGE 2: RERANKING

SURGICAL PRECISION

Use an expensive **Cross-Encoder** model to score the exact relationship between the query and each candidate.

Why Rerank?

- **BETTER THAN COSINE SIMILARITY**
- **HANDLES COMPLEX REASONING**
- **MOVES RELEVANT CHUNKS TO TOP-1**

QUERY TRANSFORMATION (HYDE)

The Problem:
User queries are short and don't look like the "answer" chunks in your database.

HYPOTHETICAL DOCUMENT EMBEDDINGS

THE HYDE WORKFLOW

1. LLM generates a "fake" hypothetical answer to the user's query.
2. The "fake" answer is embedded and used to search the vector database.
3. The most similar real documents are retrieved and fed to the LLM.

BENEFIT: SEARCHING "ANSWER-TO-ANSWER" IS MORE ACCURATE THAN "QUERY-TO-ANSWER".

HyDE helps the retriever find what the model actually needs by bridging the semantic gap between questions and answers.

THE VECTORLESS FRONTIER: PAGEINDEX

SIMILARITY ≠ RELEVANCE

REASONING-BASED NAVIGATION OVER SIMILARITY SEARCH

A NEW PARADIGM

FEATURE	VECTOR RAG	PAGEINDEX
Retrieval	Similarity (Embeddings)	Reasoning (Tree Index)
Structure	Destroyed by chunking	Preserved (Pages/Sections)
Navigation	Passive (Top-K)	Active (LLM walks the tree)

FINANCEBENCH ACCURACY

54%
NAIVE RAG

98.7%
PAGEINDEX

WHY STRUCTURE BEATS CHUNKING

FEATURE	VECTOR RAG	PAGEINDEX
RETRIEVAL	Similarity-based	Reasoning-based
CHUNKING	Arbitrary splits	Natural sections
STRUCTURE	Destroyed	Preserved
MULTI-HOP	Hard / Unreliable	Natural / Native

KEY: PAGEINDEX PRESERVES DOCUMENT STRUCTURE THAT CHUNKING DESTROYS.

THE RAG EVOLUTION SPECTRUM

FROM PASSIVE RETRIEVAL TO ACTIVE EXPLORATION

PHASE 1

NAIVE RAG

Passive Retrieval

Embed, search, and stuff. The model is a passive recipient of whatever the retriever finds.

PHASE 2

ADVANCED RAG

Smarter Retrieval

Hybrid search, reranking, and HyDE. Improving the quality of what the model sees.

PHASE 3

PAGEINDEX

Reasoning Retrieval

Vectorless, tree-based navigation. The model reasons about document structure to find answers.

PHASE 4

RLM

Active Exploration

Recursive Language Models. The model writes code to explore context as an environment.

TREND: GIVING THE MODEL MORE CONTROL OVER ITS OWN CONTEXT.